⚡ Make It Interactive

State, Input & Lists in React Native

Session 2 of 3 · 90 minutes

Recap: last week 🔁

- React Native + Expo → app on your phone
- **Components** return UI
- `View`, `Text`, `Image`
- Styling with `StyleSheet` + **Flexbox**

> Your card looked great… but it couldn't **do** anything. Let's fix that.

Today's goal 🎯

Build a **To-Do list** that you can:

- ➕ **add** tasks to (by typing)
- ✅ **tap** to mark done
- 🗑️ **delete**
- 🔢 and it shows **how many are left**

The big new idea: **state**.

Part 1

State — data that changes

What is "state"? 🧠

> **State** = data that can **change while the app runs**.
> When state changes, the screen **redraws itself** automatically.

Examples of state:

- the text you've typed
- the list of tasks
- whether a task is done

UI = a picture of your current state.

`useState` — give a component memory

```
import { useState } from 'react';

const [count, setCount] = useState(0);
//     ▲        ▲                ▲
//   value   function to     starting
//   to read  change it       value
```

- `count` → read the current value
- `setCount(...)` → change it (and redraw)

Counter demo

```
export default function App() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.big}>{count}</Text>
      <Button title="Add 1" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Tap → `setCount` → screen updates. **No manual redraw.**

⚠️ The rules of state

1. **Never change state directly** — always use the setter

```
count = count + 1;          // ❌ nothing happens
setCount(count + 1);        // ✅ updates the screen
```

2. Changing state **re-runs your component** (a "re-render")
3. Each `useState` is **independent**

Part 2

Responding to taps

Buttons & `onPress`

```
<Button title="Tap me" onPress={() => alert('Hi!')} />

// nicer-looking, fully stylable:
<TouchableOpacity onPress={handlePress}>
  <Text>Tap me</Text>
</TouchableOpacity>
```

`onPress` takes a **function** to run when tapped.

Inline vs named handlers

```
// inline (quick):
onPress={() => setCount(count + 1)}

// named (cleaner for bigger logic):
function addTask() {
  // ...several lines...
}
onPress={addTask}
```

Gotcha: `onPress={addTask}` ✅ (pass it)
`onPress={addTask()}` ❌ (runs it immediately!)

Part 3

Typing: `TextInput`

A controlled text box

```
const [text, setText] = useState('');

<TextInput
  placeholder="Add a task..."
  value={text}                  // state drives the box
  onChangeText={setText}        // typing updates state
/>
```

The input's value **is** state. Type → `onChangeText` → state → screen.

Turning typed text into a task

```
function addTask() {
  if (text.trim() === '') return;          // ignore empty
  const newTask = { id: Date.now(), title: text, done: false };
  setTasks([newTask, ...tasks]);           // add to the list
  setText('');                             // clear the box
}
```

- `Date.now()` → a quick unique **id**
- `...tasks` → keep the old tasks (next slide!)

Part 4

Lists that change

Arrays in state — copy, don't mutate

Same rule as before: make a **new array**, don't edit the old one.

```
// ➕ ADD — spread the old, add the new
setTasks([newTask, ...tasks]);

// 🗑️ REMOVE — keep everything except one
setTasks(tasks.filter((t) => t.id !== id));

// ✅ UPDATE — change one, copy the rest
setTasks(tasks.map((t) =>
  t.id === id ? { ...t, done: !t.done } : t
));
```

`.filter` and `.map` in plain words

```
// filter → KEEP the ones that pass the test
[1,2,3,4].filter((n) => n > 2)        // [3, 4]

// map → TRANSFORM every item
[1,2,3].map((n) => n * 10)            // [10, 20, 30]
```

- **delete** a task → `filter` it out
- **toggle** a task → `map`, flip the matching one

Showing a list: `FlatList`

```
<FlatList
  data={tasks}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => (
    <Text>{item.title}</Text>
  )}
/>
```

- `data` → your array
- `renderItem` → how to draw **one** item
- `keyExtractor` → a unique id per row (helps performance)

Empty states matter

```
<FlatList
  data={tasks}
  ListEmptyComponent={
    <Text style={styles.empty}>No tasks yet. Add one! ✨</Text>
  }
  ...
/>
```

Always tell the user when there's **nothing to show**.

Part 5

Reusable components & props

Extract a `<TodoItem>` component

```
function TodoItem({ task, onToggle, onDelete }) {
  return (
    <View style={styles.item}>
      <TouchableOpacity onPress={onToggle}>
        <Text>{task.done ? '✅' : '⬜'} {task.title}</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={onDelete}>
        <Text>🗑️</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Props flow down

```
<FlatList
  data={tasks}
  renderItem={({ item }) => (
    <TodoItem
      task={item}
      onToggle={() => toggleTask(item.id)}
      onDelete={() => deleteTask(item.id)}
    />
  )}
/>
```

Parent owns the data & functions → passes them to each child.

Derived values (no extra state!)

```
const remaining = tasks.filter((t) => !t.done).length;

<Text>{remaining} left to do</Text>
```

Don't store what you can **calculate** from existing state.

🎬 Live-code target

The full **To-Do app**:

header with counter → input + add button → scrollable list of tappable, deletable tasks → empty state.

🧪 Activity 2

Build the To-Do app

Time: ~25 min · **File:** `App.js`

Make it work end-to-end:

1. A `text` state + a **`TextInput`** (controlled)
2. A `tasks` state (start with 1–2 sample tasks)
3. **Add** a task (`...spread`), **clear** the input
4. **Toggle** done (`map`) and **delete** (`filter`)
5. Show them in a **`FlatList`** with an empty state

Activity 2 — checkpoints & stretch

✅ **Done when:** you can add, complete (tap), and delete tasks, and the "left to do" count is correct.

🌟 **Stretch goals:**

- A **"Clear done"** button (`filter` out completed)
- Strike-through style on completed tasks
- Prevent adding **empty** tasks (already handled — read how!)
- Sort done tasks to the bottom

⚠️ But there's a problem…

Close the app. Reopen it.

Your tasks are gone. 😱

✅ Recap — you can now…

- Store changing data in **state** (`useState`)
- Respond to taps with **`onPress`**
- Capture typing with a controlled **`TextInput`**
- Add / remove / update with **spread / filter / map**
- Render dynamic lists with **`FlatList`**
- Build **reusable components** with **props**

🏠 Take-home

Add a **"Clear done"** button and a **strike-through** style for completed tasks.

⏭️ Next session

We make your tasks **survive a restart** with a real on-device **SQLite database** — full **CRUD**. 💾